

Learned In-Betweeners Make Simulated Dexterous Hands Learn Faster

Anonymous authors (double-anonymous review)

Abstract—Modular motion models such as MotionBricks build motion from reusable parts, which makes them attractive for dexterous hands, whose tasks are long and whose demonstrations are scarce. Unfortunately, prior work on synergies, latent action spaces and skill chaining changes several things at once, so it is unclear where a modular gain comes from. We test each of MotionBricks’ two layers on simulated hands, with the main comparisons registered before they ran. At the motion level, a keyframe action space, target poses joined by a learned in-beweenner, leads a continuous latent that decides at the same rate at the registered checkpoint (0.59 against 0.22 success on held-out skill sequences at 1.5 M frames; difference +0.38, 95% CI 0.23 to 0.52) and learns faster, reaching its training plateau sooner. Other modular action spaces, straight-line keyframes and a bank of fixed primitives, do not. At 5 M frames the data do not separate it from the latent. At the skill level, selecting the best policy for each stage completes 97 of 400 test episodes against 61 for the best single policy, in one run of the selection pipeline; training the skills separately does not help (45 of 400). The gains come from a learned in-beweenner and from per-stage selection, not from training the skills separately.

Index Terms—Dexterous manipulation, reinforcement learning, learning from demonstration, multifingered hands, deep learning methods.

I. INTRODUCTION

Dexterous manipulation tasks are long and their demonstrations are scarce, so a learner that reuses parts is attractive. MotionBricks [1] generates character motion from reusable parts at two layers: a latent in-betweening backbone that joins target keyframes, and smart primitives that generate those keyframes from commands. Modularity of this kind promises a hand that learns faster, because each part is simpler than the whole task, and performs better, because parts recombine into behavior never seen whole.

Whether modularity keeps these promises in control is unclear. Synergy, latent and dynamical-system action spaces [2]–[4] and pretrained behavioral priors [5] report faster learning, and skill chaining works on dexterous hands when the hand-offs between skills are trained explicitly [6]. Each of these changes the prior, the decision rate, the data or the training procedure together, so it is unclear which part of the design produces the gain.

This paper separates the parts. A flat policy that knows which stage it is in can, with enough capacity, represent what a modular policy represents, so a modular gain is expected in how a solution is found rather than in which solutions exist. We therefore test two routes by which modularity changes the search: an action space built from poses and a learned in-beweenner, compared with a continuous latent at the same decision rate; and modular deployment, which lets each stage’s policy be selected on its own, compared with separate training on the same data.

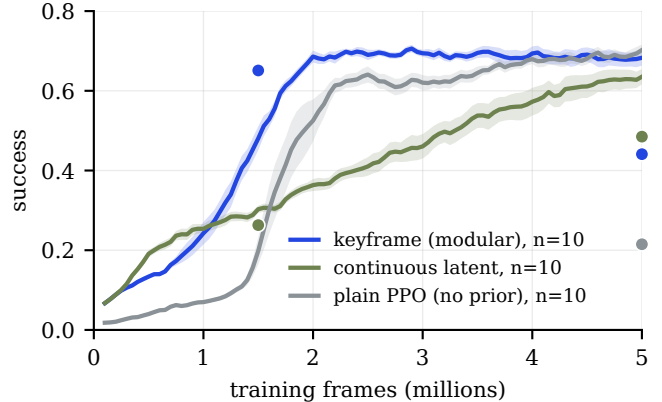


Fig. 1. Keyframes with a learned in-beweenner learn faster. On the training sequences the keyframe interface (blue) reaches its plateau by 2 M frames, while plain PPO (grey) and the continuous latent (green) take longer; by 5 M frames the three are within 0.06 of each other. On held-out sequences (dots) it leads the latent at 1.5 M frames; at 5 M the two are not distinguishable. Lines: training success, mean and standard error over 10 RL seeds; dots: the same seeds (Section IV reports all 20 seeds at 1.5 M).

Both routes pay. At the motion level, a keyframe interface in MotionBricks’ sense leads a continuous latent that decides at the same rate on held-out skill sequences at 1.5 M frames (+0.38) and reaches its training plateau sooner; by 5 M frames the data do not separate the two (Fig. 1). At the skill level, selecting the best policy for each stage completes more episodes than the best single policy; training the skills separately does not.

Our contributions are:

- 1) **Faster learning from a learned in-beweenner.** At the same decision rate, keyframes joined by a learned in-beweenner lead a continuous latent at the registered checkpoint and reach the training plateau sooner than the latent and plain PPO (post hoc); straight-line keyframes and fixed primitives do not (Section IV).
- 2) **Per-stage selection, not separate training.** A design that separates the two shows that selecting each stage’s policy on its own raises success in our run, and that separate training on the same demonstrations does not (Section V).

II. RELATED WORK

Priors as action spaces. A learned motion prior turns a high-dimensional hand into a smaller search space. LAMP [7] uses a continuous latent prior as the action space for residual RL, and DexSynRefine [8] grounds motion primitives with residual RL. Synergy, latent and dynamical-system action spaces report faster learning [2]–[4], and

pretrained behavioral priors do as well [5]. Several also raise final success, most clearly where joint-space RL fails to learn some objects within the same budget [9] or meets a severe exploration problem [10]. Learned skill priors act through fixed-length skills [11], and action chunking predicts several actions at once [12]; both shorten the decision horizon, as a keyframe interface does. What this line of work leaves open is how much of the speed-up is the structure of the prior and how much is the decision rate; the comparison here holds the decision rate fixed.

Hierarchy and skill chaining. Options [13], learned end to end [14], and goal-conditioned two-level hierarchies [15] formalize policies that act through sub-policies. Modular motion appears as authored primitives over a latent backbone [1], switching dynamical systems [16], [17] and mode-adaptive gating [18]. Skill chaining trains each skill to reach the initiation set of the skill that follows it [19], terminal-state regularization helps the next skill take over in arm manipulation [20], and on dexterous hands chaining succeeds when the hand-offs between skills are trained explicitly [6]. These methods add machinery at the hand-offs, so they do not isolate what training modules separately contributes. Analyses of hierarchical RL trace most of its benefit in fully observed tasks to exploration [21] and its efficiency to sub-problems that repeat [22], which points to the search.

Policy selection. Imitation-learned policies vary widely across training checkpoints, and common selection rules can fall far short of the best checkpoint [23]; with a limited evaluation budget, which trained policy is selected can change which method appears best [24]. That work selects whole policies. Chaining pipelines train each skill as a separate policy [6], [20] but do not choose among several trained candidates for each stage. On the relocate task used here, DAPG [25] fine-tunes a behavior-cloned policy with a demonstration-augmented policy gradient; our skill level uses behavior cloning alone, to isolate selection from training.

III. HAND-MOTION PRIORS

All priors are trained on OakInk [26] hand motion, represented as 27 anatomical degrees of freedom (four per finger, five for the thumb, six for the wrist) projected from MANO [27]. The *continuous latent* is a variational autoencoder ($\beta = 1$ in the notation of [28], i.e., a standard VAE, trained with free bits [29]) that encodes a 32-frame window into a 12-D latent per frame, with GRU encoder and decoder (2.28 M parameters) and a bounded output, so joint limits hold by construction. The *keyframe in-between*, the MotionBricks-style prior, is a GRU trained on the same motion to fill the frames between two target poses. The *clip bank* is a bank of $K = 16$ fixed motion primitives mixed softly with a style code.

IV. MOTION LEVEL: A LEARNED IN-BETWEENER LEARNS FASTER

Interfaces. Each prior becomes the action space of PPO [30], and each prior interface takes one decision every 16 frames. (a) Clip bank: a soft mixture of primitives and

a style. (b) Continuous latent: the 12-D latent, held for 16 frames. (c) Keyframe, as in MotionBricks’ in-betweening backbone [1]: a target pose, reached by the learned in-betweener. (d) The same target pose reached by straight-line interpolation, with no learned prior. Each adds a per-frame residual of at most 3° , capped at $\pm 15^\circ$ in total. (e) Plain PPO sets joint targets every frame, with no prior. Every policy observes a one-hot code of the current skill and the fraction of the skill elapsed.

Task. On the Shadow Hand E3M5 [31] in MuJoCo [32], a 5 cm, 0.20 kg cube rests on the palm while a lateral acceleration of 1.5 g acts on it, as if the palm were tilted. An episode is a sequence of skills of forty frames each: hold with the acceleration along x , hold with it along y , release. Policies train on the four two-skill sequences and are scored on the two three-skill sequences they never see, over 100 episodes with the deterministic policy; an episode succeeds when the cube is held at the end of every hold and has left the hand at the end of the release.

Training. PPO runs 16 parallel environments with rollouts of 256 steps, batches of 1024, learning rate 3×10^{-4} , entropy coefficient 0.01, 10 epochs per update, $\gamma = 0.99$, $\lambda = 0.95$ and clip 0.2. Actor and critic have two hidden layers of 256 units; exploration is state-dependent, resampled every 8 steps, from an initial log standard deviation of -1 . The reward is one unit for every frame the cube is held during a hold, and a graded per-frame reward during release. Every interface uses the same settings; the interfaces share the decision rate but differ in action dimension and in the scale of their exploration noise.

Faster. At 1.5 M frames the keyframe interface succeeds on 0.59 of held-out sequences against the latent’s 0.22 (20 seeds; difference $+0.38$, 95% CI $[0.23, 0.52]$, t -interval over seeds; 17 of 20 seeds, sign test $p = 0.003$, as declared), and the lead holds on longer unseen sequences of four and five skills (0.51 and 0.48 against 0.22 and 0.15). Fig. 1 shows the same lead on the training sequences: the keyframe interface reaches its plateau of 0.69 by 2 M frames, plain PPO comes within 0.05 of it at 2.5 M, and the latent is still below it at 5 M (0.64). Over the first 2.5 M frames its mean training success is 0.40, against 0.25 for plain PPO and 0.27 for the latent (10 seeds each, Mann–Whitney $p = 10^{-4}$ for both, post hoc). Because the latent also takes one decision every 16 frames, the difference is not the decision rate. The same target poses reached by straight lines never learn to release (0 of 10 seeds), so on this task the learned in-betweener, not the keyframes alone, carries the gain. Not every modular action space helps: the bank of fixed primitives is the weakest prior (0.08), so the gain belongs to the learned in-betweener rather than to modularity as such. Fig. 2 shows one held-out episode.

The end point. At 5 M frames (seeds 0–9) the data do not separate the keyframe interface from the latent on held-out sequences (Table I; 0.44 against 0.49, paired difference -0.04 , 95% CI $[-0.22, 0.13]$, inconclusive). It still beats plain PPO there (0.44 against 0.22; difference $+0.23$, 95% CI $[0.09, 0.36]$; 9 of 10 seeds, as declared), but plain PPO

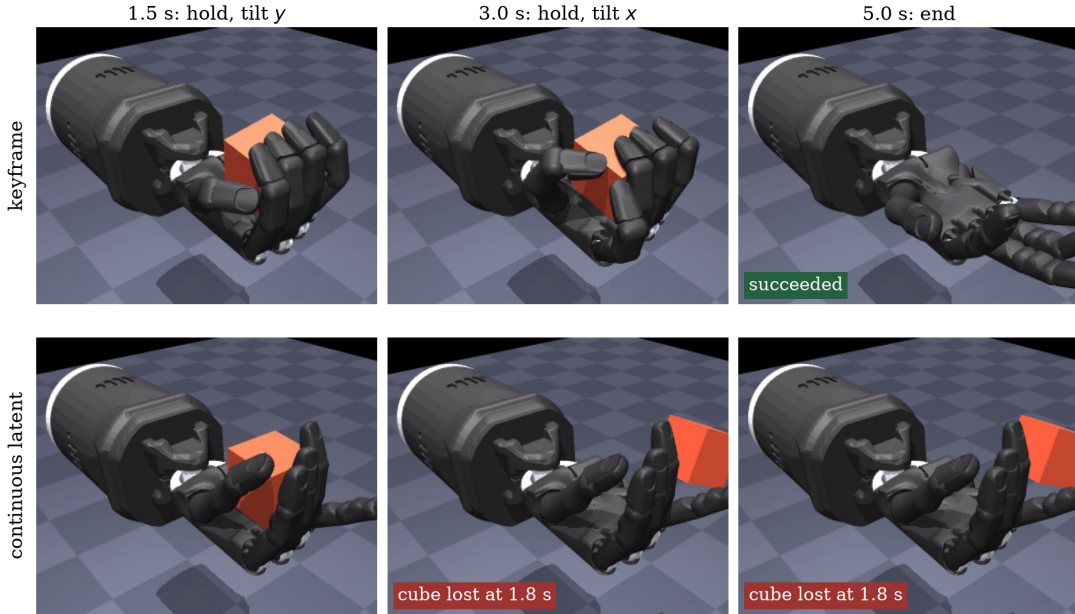


Fig. 2. The keyframe interface holds the cube through an unseen sequence; the continuous latent drops it. One held-out sequence at 1.5 M frames (hold under a y tilt, hold under an x tilt, release); top, the keyframe interface; bottom, the continuous latent, which loses the cube during the first hold. RL seed 13, whose success is closest to the 20-seed means; the first evaluation episode (of 100) in which the keyframe interface succeeds and the latent fails.

TABLE I
SUCCESS ON UNSEEN SKILL SEQUENCES BY ACTION SPACE

interface	1.5 M frames	5 M frames
plain PPO (no prior)	–	0.22
continuous latent	0.22	0.49
keyframe, in-betweeners	0.59	0.44
keyframe, straight-line	0.00	–
$K = 16$ clip bank	0.08	–

Seeds: 20 at 1.5 M; 10 for the straight-line and bank rows and at 5 M.

also decides every frame, so this comparison changes the decision rate as well as the prior. On the training sequences the three end within 0.06 of each other. Longer training does not help held-out success: on the same seeds the keyframe interface falls from 0.65 at 1.5 M to 0.44, as both learned priors settle on holding perfectly (0.96 to 1.00 per hold on the training sequences) and releasing less than half the time. On three other objects (a heavier cube, a sphere and a cylinder) the early lead is smaller and not significant (pooled test, $p = 0.067$).

V. SKILL LEVEL: SELECTION, NOT SEPARATE TRAINING

Task. MotionBricks’ other layer composes smart primitives; the skill level asks the matching question for units that act, namely whether chaining separately trained skills pays. The test uses Adroit’s relocate [25] (as packaged in Gymnasium-Robotics 1.4.2), extended into a sparse multi-stage task: grasp the ball from the table, then carry it to a random target in the air, once or twice. The data are the 25 human demonstrations of relocate in D4RL [33]; we split each into one grasp and one carry, and every

policy is behavior-cloned from them; PPO from scratch never completes a carry (5 M steps, two seeds). A modular policy trains one skill per stage and chains them with a fixed sequencer; a *flat* policy is one policy for the whole task, with the stage index in its input. A stage counts as done only after its goal holds for 25 consecutive steps: with a one-step criterion, carry policies learn to fling the ball through the target, leaving an end state the next skill cannot start from, the kind of failure that motivates terminal-state regularization [20].

Design. The design separates selection from training, on fresh seeds and 400 test episodes never used for any choice, with 20 trainings per side. Variant A is the best of 20 flat policies, used for every stage. Variant B takes the same 20 flat policies and uses the best grasper for the grasp and the best carrier for every carry. Variant C trains 10 grasp skills (on the grasp segments of the demonstrations) and 10 carry skills (on the carry segments) and chains the best of each. A against B changes only the selection; B against C changes only the training. Every choice uses 200 validation episodes per stage: graspers start from the benchmark’s reset, and carriers start from a fixed bank of states in which a separate grasp policy holds the ball. Behavior cloning regresses the policy’s mean action onto the demonstrated actions (mean squared error, Adam, learning rate 10^{-3} , batch 256, 20 epochs), with the same network as above and observation normalization fitted to the demonstrations. An earlier registered comparison with five seeds per variant is at the floor at its declared length of four stages ($p = 0.45$); at two stages the modular variant wins (38 against 8 of 200, $p < 10^{-5}$), which we traced after the fact to per-stage selection. This design tests that reading.

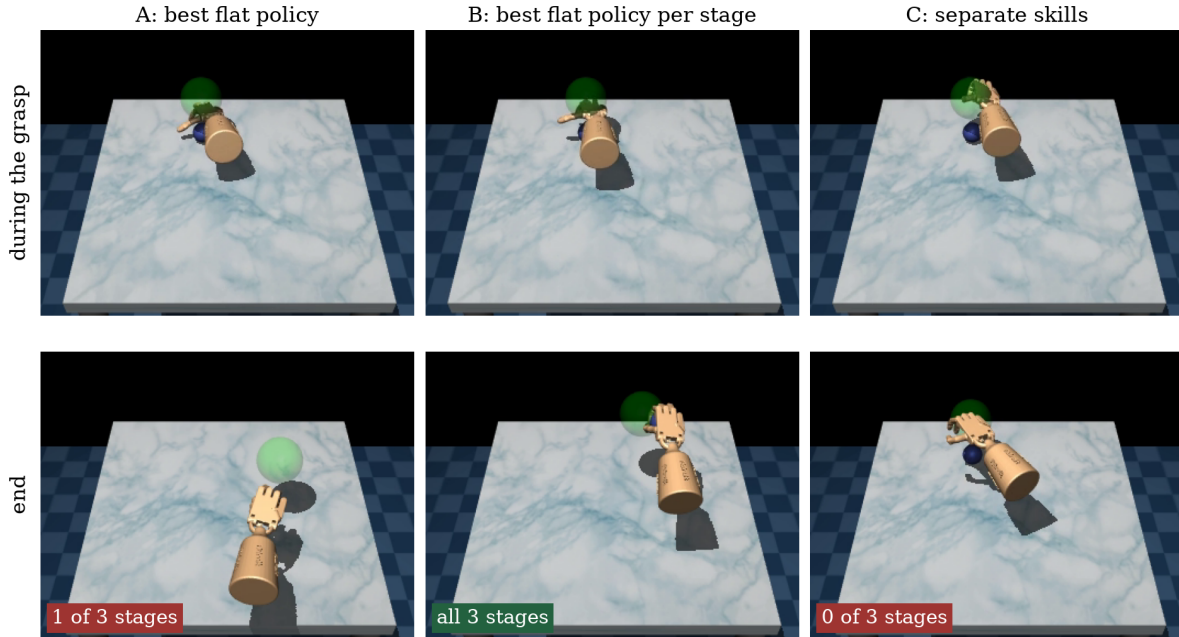


Fig. 3. Selecting per stage completes the task where the other variants stop. Same test start for the three variants (grasp, carry, carry again); top, during the grasp; bottom, at the end. B, the best flat policy for each stage, completes all three stages; A grasps the ball but never delivers it, and C never completes the grasp. One test episode, the first in which B completes every stage; B does so in 47 of 400 test episodes (Table II).

TABLE II

SELECTION, NOT TRAINING (400 TEST EPISODES, PRE-REGISTERED)

variant	grasp	carry grasp	2 stages	3 stages
A: best flat policy	0.77	0.20	61	12
B: best flat per stage	0.72	0.34	97 ($p = 0.002$)	47 ($p < 10^{-5}$)
C: separate skills	0.57	0.20	45 ($p = 0.12$)	6 ($p = 0.23$)

Per-stage rates for the two-stage task (carry | grasp: carries completed after a completed grasp); the last two columns count episodes that complete every stage. Fisher’s exact test against A, over test episodes for the selected combination; the pipeline runs once.

Per-stage selection helps. In this run of the selection pipeline, selecting the best policy for each stage beats the best single policy: 97 against 61 of 400 episodes at two stages ($p = 0.002$) and 47 against 12 at three ($p < 10^{-5}$). It wins through the carry: B’s carrier completes 0.34 of carries after a grasp, against A’s 0.20.

Separate training does not. The registered prediction, that separately trained skills beat the best flat policy, fails (45 against 61, $p = 0.12$), and separate training loses to per-stage selection (45 against 97, $p < 10^{-5}$). C loses at the grasp (0.57 against 0.77). Without selection, the declared comparison over seeds finds separately trained carry skills worse than the carry stage of flat policies (0.14 against 0.27 on validation, $p < 10^{-5}$) and grasp skills not distinguishable ($p = 0.19$); on the test episodes both are lower (grasp 0.14 against 0.23, carry 0.17 against 0.26, post hoc).

Not a matter of pool size. B chooses from 20 policies per stage and C from 10, and each variant is one selected combination. A post hoc follow-up, with its analysis written

down before it ran, scores every selection rule by the product of its per-stage test rates. The product assumes a carrier’s success does not depend on which grasper precedes it; for the three selected combinations it is within 0.05 of the chained test result (A 0.20 against 0.15, B 0.26 against 0.24, C 0.10 against 0.11). Restricted to each of the 184 756 possible pools of 10 flat policies, per-stage selection still beats separate training in 99% of pools (mean 0.23 against 0.10). Over 10 000 redrawn pools of 10 per side, it beats separate training in 96% of draws and whole-policy selection in 82% (Fig. 4). These pools are subsets of the same 20 policies scored on the same test episodes, so they show that the result does not hinge on pool size, not that it replicates; repeating the pipeline on independent seeds would test that.

VI. DISCUSSION: WHERE THE GAINS COME FROM

Both results point to the same place. Keyframes with a learned in-between pay by exploring faster; modular deployment pays by letting each stage’s policy be chosen on its own. Neither requires training modules separately, and at the skill level separate training costs skill. This is consistent with the finding that most of the benefit of hierarchy comes from exploration [21], and with synergy, latent and dynamical-system action spaces, which report faster learning [2]–[4]. The speed-up depends on the prior: a learned in-between explores well, while straight-line keyframes and a bank of fixed primitives do not.

Per-stage selection works because each stage is validated on its own. With N trained policies, per-stage selection picks the best of each stage, where whole-policy selection picks only the best product. If a carrier’s success does not

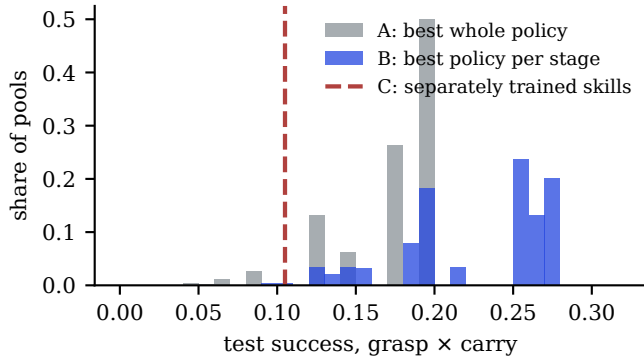


Fig. 4. Selection, not pool size. With pools matched at 10 policies per stage, per-stage selection (B, blue) lies above separate training (C, dashed line) in 99% of pools and mostly above whole-policy selection (A, grey). Each bar counts pools of 10 flat policies (184 756 in all) by the product of per-stage test rates (grasp $\times$ carry), with every choice made on validation.

depend on which grasper precedes it, the product of the per-stage best is never below the best whole policy; the chained results above stay close to this product. Selection is therefore expected to help; what the experiment measures is how much, and that separate training does not add to it. Chaining pipelines train each skill as a separate policy [6], [20] but do not choose among several trained candidates for each stage; the result here measures what that choice contributes on its own, and shows that it, not separate training, carries the gain.

When would separately trained modules pay as well? A chain of modules succeeds with probability $\prod_i P(\text{module } i \mid \text{end states of module } i-1)$, so the cost sits at the hand-offs. Four conditions, formulated during this study and offered as hypotheses, state when separate modules would pay: (i) the solution lies inside what the modules express; (ii) hand-offs are clean; (iii) there is something to amortize, such as a horizon a flat learner cannot cross or combinations it has not seen; and (iv) a flat learner given the same data does not get the same benefit. Table III maps the two tasks onto them; neither meets (iv). MotionBricks, in animation, plausibly meets all four: its goal is human-like motion and its data are human motion, it is kinematic so any pose starts the next brick, one library serves many applications, and it has over 3.5×10^5 clips.

VII. LIMITATIONS AND FUTURE WORK

Everything is in simulation, with one task per layer, and the early lead is significant on the cube only. The interfaces share one set of PPO settings, and the skill-level selection runs once, with behavior cloning from 25 demonstrations. Stronger baselines (learned skill priors [11], action chunking [12] adapted to RL, T-STAR [20]), interface-specific tuning and repeated selection pipelines are the natural next steps.

VIII. CONCLUSION

On these simulated dexterous hands, a MotionBricks-style keyframe interface with a learned in-betweeners learns faster than a continuous latent at the same decision rate, and the

TABLE III
THE FOUR CONDITIONS FOR SEPARATELY TRAINED MODULES TO PAY, AGAINST THE TWO TASKS. NEITHER TASK MEETS (IV).

condition	motion level	skill level
(i) solution inside	partly: the grip needs 83° of mean finger flexion; the most-flexed grips we measure in OakInk [26] and TACO [34] reach $73\text{--}76^\circ$; the residual covers the gap	yes: skills learned on the task
(ii) clean hand-offs	yes: poses only	only when enforced: a one-step stage lets carry fling the ball
(iii) something to amortize	partly: faster exploration, one task	yes: PPO from scratch never carries
(iv) flat cannot match	no: ends level with the latent	no: flat policies selected per stage do best

two end at levels the data do not separate. At the skill level, selecting the best policy for each stage raises success, while training the skills separately does not. The gains come from a learned in-betweeners and from per-stage selection, not from training the skills separately.

ACKNOWLEDGMENT

Generative AI (Claude, Anthropic) was used under the authors' direction to draft and revise the text of all sections, to write the analysis and figure scripts behind Sections IV and V, and to launch the experiment runs. The authors designed the study and take full responsibility for the content. Every number in the paper is computed by scripts from the run artifacts; the code, the artifacts and the pre-registration log will be released on acceptance.

REFERENCES

- [1] T. Wang *et al.*, "MotionBricks: Scalable real-time motions with modular latent generative model and smart primitives," *ACM Trans. Graph.*, vol. 45, no. 4, Art. no. 85, pp. 1–22, 2026, doi: 10.1145/3811334.
- [2] C. Berg, V. Caggiano, and V. Kumar, "SAR: Generalization of physiological dexterity via synergistic action representation," in *Proc. Robot.: Sci. Syst.*, 2023, doi: 10.15607/RSS.2023.XIX.007.
- [3] E. Y. Puang, F. Ceola, G. Pasquale, and L. Natale, "PCHands: PCA-based hand pose synergy representation on manipulators with N-DoF," in *Proc. IEEE-RAS Int. Conf. Humanoid Robots*, 2025, pp. 475–482, doi: 10.1109/Humanoids65713.2025.11203193.
- [4] S. Bahl, M. Mukadam, A. Gupta, and D. Pathak, "Neural dynamic policies for end-to-end sensorimotor learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 5058–5069.
- [5] V. Caggiano, S. Dasari, and V. Kumar, "MyoDex: A generalizable prior for dexterous manipulation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 3327–3346.
- [6] Y. Chen, C. Wang, L. Fei-Fei, and C. K. Liu, "Sequential dexterity: Chaining dexterous policies for long-horizon manipulation," in *Proc. Conf. Robot Learn.*, 2023, pp. 3809–3829.
- [7] X. Yang *et al.*, "LAMP: Latent motion prior-guided real-world learning for dexterous hand manipulation," 2026, arXiv:2607.06323.
- [8] H. Lee, H. Jung, S.-H. Heo, and S. Yang, "DexSynRefine: Synthesizing and refining human-object interaction motion for physically feasible dexterous robot actions," 2026, arXiv:2605.05925.
- [9] P. Rivera, E. Valarezo Añazco, and T.-S. Kim, "Object manipulation with an anthropomorphic robotic hand via deep reinforcement learning with a synergy space of natural hand poses," *Sensors*, vol. 21, no. 16, Art. no. 5301, 2021, doi: 10.3390/s21165301.

- [10] Z. Luo, J. Cao, S. Christen, A. Winkler, K. Kitani, and W. Xu, "OmniGrasp: Grasping diverse objects with simulated humanoids," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 2161–2184.
- [11] K. Pertsch, Y. Lee, and J. J. Lim, "Accelerating reinforcement learning with learned skill priors," in *Proc. Conf. Robot Learn.*, 2020, pp. 188–204.
- [12] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robot.: Sci. Syst.*, 2023, doi: 10.15607/RSS.2023.XIX.016.
- [13] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, no. 1–2, pp. 181–211, 1999, doi: 10.1016/S0004-3702(99)00052-1.
- [14] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, 2017, doi: 10.1609/aaai.v31i1.10916.
- [15] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018.
- [16] S. W. Linderman, M. J. Johnson, A. C. Miller, R. P. Adams, D. M. Blei, and L. Paninski, "Bayesian learning and inference in recurrent switching linear dynamical systems," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2017, pp. 914–922.
- [17] Z. Dong, B. A. Seybold, K. P. Murphy, and H. H. Bui, "Collapsed amortized variational inference for switching nonlinear dynamical systems," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2638–2647.
- [18] H. Zhang, S. Starke, T. Komura, and J. Saito, "Mode-adaptive neural networks for quadruped motion control," *ACM Trans. Graph.*, vol. 37, no. 4, Art. no. 145, 2018, doi: 10.1145/3197517.3201366.
- [19] G. Konidaris and A. G. Barto, "Skill discovery in continuous reinforcement learning domains using skill chaining," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 22, 2009.
- [20] Y. Lee, J. J. Lim, A. Anandkumar, and Y. Zhu, "Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization," in *Proc. Conf. Robot Learn.*, 2021, pp. 406–416.
- [21] O. Nachum, H. Tang, X. Lu, S. Gu, H. Lee, and S. Levine, "Why does hierarchy (sometimes) work so well in reinforcement learning?" presented at the NeurIPS Deep RL Workshop, 2019, arXiv:1909.10618.
- [22] Z. Wen, D. Precup, M. Ibrahimi, A. Barreto, B. Van Roy, and S. Singh, "On efficiency in hierarchical reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 6708–6718.
- [23] A. Mandlekar *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," in *Proc. Conf. Robot Learn.*, 2021, pp. 1678–1690.
- [24] V. Kurenkov and S. Kolesnikov, "Showing your offline reinforcement learning work: Online evaluation budget matters," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 11729–11752.
- [25] A. Rajeswaran *et al.*, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," in *Proc. Robot.: Sci. Syst.*, 2018, doi: 10.15607/RSS.2018.XIV.049.
- [26] L. Yang *et al.*, "OakInk: A large-scale knowledge repository for understanding hand-object interaction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 20953–20962.
- [27] J. Romero, D. Tzionas, and M. J. Black, "Embodied hands: Modeling and capturing hands and bodies together," *ACM Trans. Graph.*, vol. 36, no. 6, Art. no. 245, 2017, doi: 10.1145/3130800.3130883.
- [28] I. Higgins *et al.*, " β -VAE: Learning basic visual concepts with a constrained variational framework," in *Proc. Int. Conf. Learn. Represent.*, 2017.
- [29] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, "Improved variational inference with inverse autoregressive flow," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, arXiv:1606.04934.
- [30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, arXiv:1707.06347.
- [31] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors, "MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo," 2022. [Online]. Available: https://github.com/google-deeppmind/mujoco_menagerie
- [32] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2012, pp. 5026–5033.
- [33] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for deep data-driven reinforcement learning," 2020, arXiv:2004.07219.
- [34] Y. Liu *et al.*, "TACO: Benchmarking generalizable bimanual tool-action-object understanding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21740–21751.